

I. INTRODUCTION: WHY CALCULUS IS THE "VERB" OF AI

If Linear Algebra is the "Noun" (describing what data *is*), then Calculus is the "Verb" (describing how data *changes*). In Artificial Intelligence, we don't tell the computer exactly how to solve a problem. Instead, we give it a **Loss Function** (an error score) and tell it to "go down" until the error is as small as possible.

Calculus provides the mathematical tools to find the **direction** and **speed** of that descent. Without Calculus, a Neural Network would be a static set of numbers that never learns.

II. THE DERIVATIVE: MEASURING SENSITIVITY

The fundamental unit of Calculus is the **Derivative** $f'(x)$. It measures the rate of change of a function.

2.1. The "Weight Sensitivity" Concept

In a Neural Network, every weight (w) is a tiny knob we can turn. The derivative of the Loss function ($\mathcal{L}$) with respect to a weight (w), written as

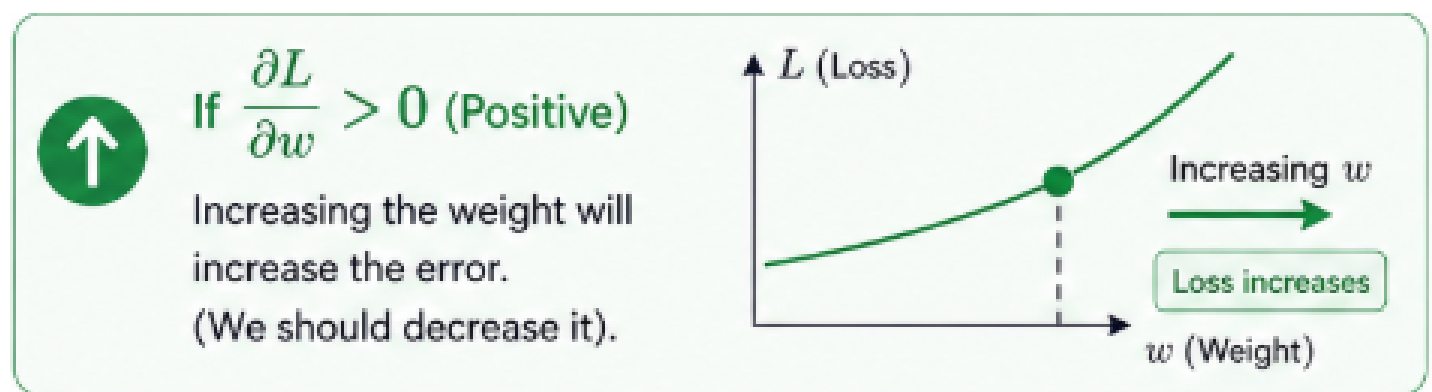
$$\frac{\partial \mathcal{L}}{\partial w},$$

tells us:

- If

$$\frac{\partial \mathcal{L}}{\partial w}$$

is **Positive**: Increasing the weight will increase the error. (We should decrease it).



- If

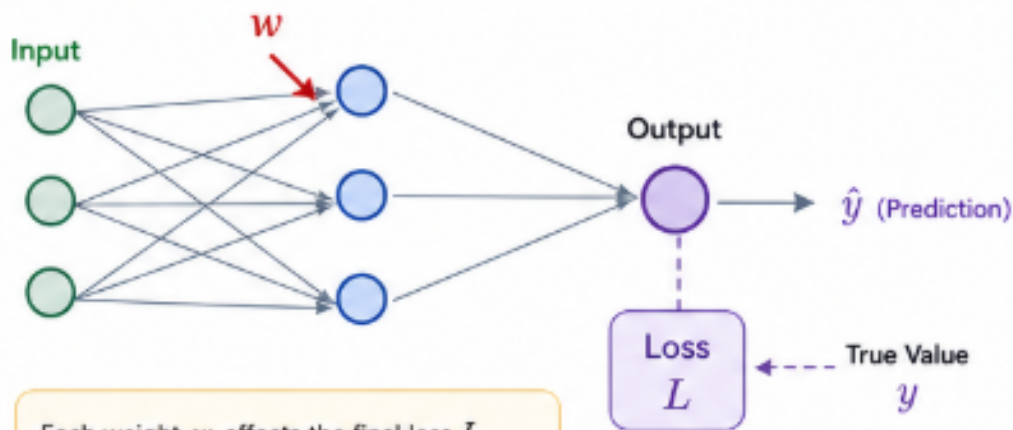
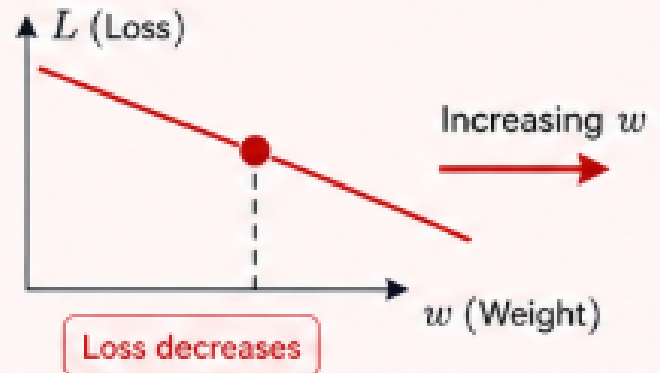
$$\frac{\partial \mathcal{L}}{\partial w}$$

is **Negative**: Increasing the weight will decrease the error. (We should increase it).



If $\frac{\partial L}{\partial w} < 0$ (Negative)

Increasing the weight will decrease the error.
(We should increase it).



Each weight w affects the final loss L .

The derivative $\frac{\partial L}{\partial w}$ tells us how **sensitive** the loss is to a small change in that weight.



Key Takeaway

$\frac{\partial L}{\partial w}$ is the "sensitivity" of the loss to that weight.

- It tells us the **direction** to move the weight to reduce the error.
- It tells us how **strongly** that weight influences the error.

2.2. Partial Derivatives and the Gradient (



)

Modern AI models have millions of weights. We calculate a **Partial Derivative** for each weight individually while holding others constant. When we group all these partial derivatives into a single

vector, we get the **Gradient**



L).

- The Gradient always points in the direction of the **steepest increase** in error.
- To learn, the AI must move in the **opposite** direction -



- L).

III. THE CHAIN RULE: THE BACKBONE OF BACKPROPAGATION

Deep Learning models are "deep" because they consist of many layers stacked together. Mathematically, this is a **Composite Function**:

$$\text{Output} = f(g(h(\text{Input})))$$

3.1. Propagating the Error

To find out how a weight in the *first* layer affects the *final* error, we cannot calculate it directly. We use the **Chain Rule**:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

In AI, this allows the error signal at the output layer to "flow backward" through every single hidden layer, multiplying derivatives along the way. This is why the primary training algorithm is called **Backpropagation**.

IV. OPTIMIZATION: SEARCHING FOR THE GLOBAL MINIMUM

The goal of training is to find the **Global Minimum**—the lowest point in the "Valley of Errors."

4.1. Gradient Descent (GD)

The most famous optimization algorithm. It updates the weights using the formula:

$$w_{new} = w_{old} - \eta \cdot \nabla L$$

•

η

(Learning Rate): The size of the step we take.

- If

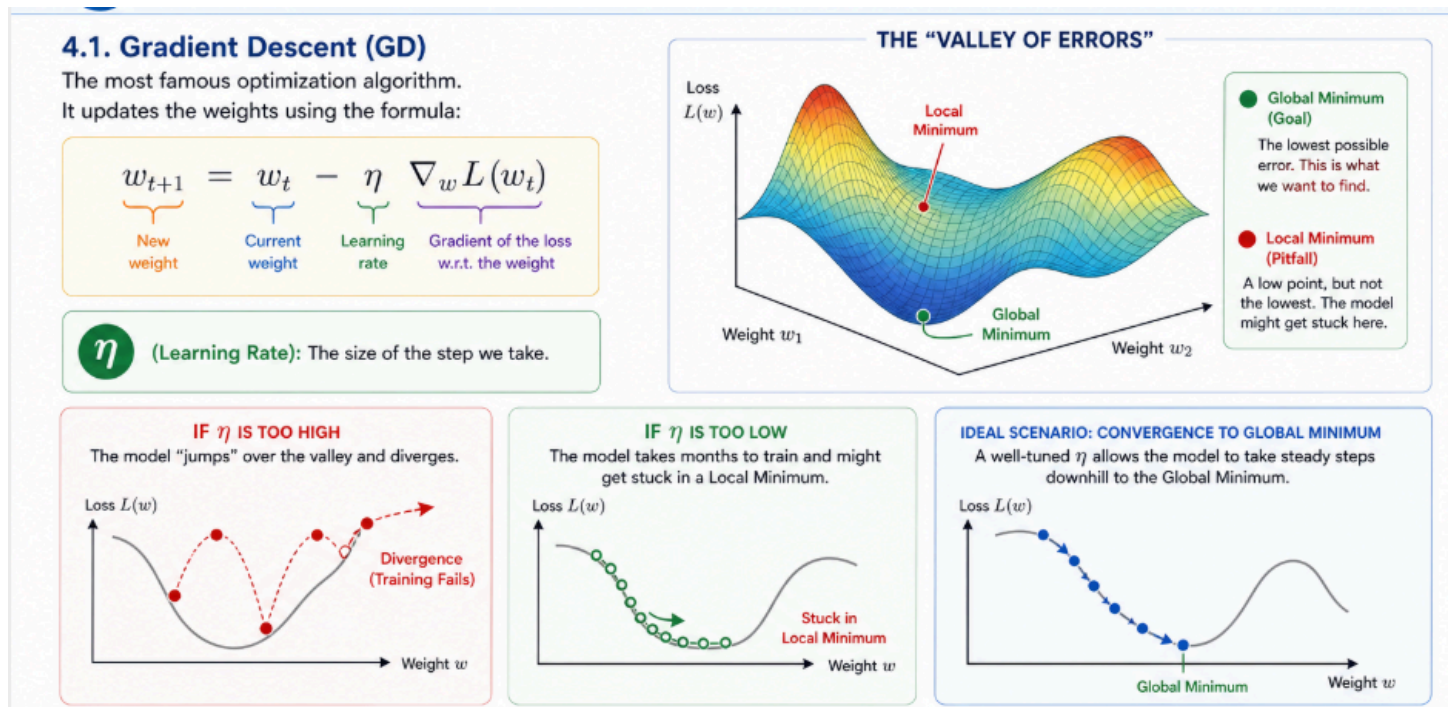
η

is **too high**: The model "jumps" over the valley and diverges.

- If

η

is **too low**: The model takes months to train and might get stuck in a **Local Minimum**.



4.2. Advanced Optimizers

Standard GD is often too slow. Modern AI uses advanced variants:

- **Stochastic Gradient Descent (SGD):** Updates weights using only one random sample at a time (much faster for Big Data).
- **Adam (Adaptive Moment Estimation):** The industry standard. It automatically adjusts the learning rate for each individual weight based on past gradients.

V. PRACTICAL LAB: GRADIENT CALCULATION (PYTORCH)

PyTorch uses "Autograd" to handle all the calculus for us automatically.

Python

```
import torch
```

1. Create a weight tensor and tell PyTorch to track gradients

```
w = torch.tensor([2.0], requires_grad=True)
```

```
x = torch.tensor([5.0])
```

2. Define a simple Loss Function: $L = (w \cdot x - 10)^2$

If $w=2$ and $x=5$, $w \cdot x$ is 10, so Loss is 0.

Let's see what happens if the weight starts at 3.0.

```
w = torch.tensor([3.0], requires_grad=True)
```

```
loss = (w * x - 10)**2
```

3. Trigger Backpropagation (The Calculus)

```
loss.backward()
```

4. Access the Gradient: $dLoss/dw$

Formula: $2 * (w \cdot x - 10) * x = 2 * (15 - 10) * 5 = 50$

```
print(f'Gradient at w=3.0 is: {w.grad.item()}')
```

Result 50 means increasing 'w' by 1 would increase 'loss' by roughly 50.